

# AI ASSISTED CODING

## Assisgnment-14.4

**Name:** v.praneeth

**Batch:** 41

**Roll No:** 2303A52146

### Task 1:

#### Responsive Homepage Layout for a Startup Website

#### Scenario:

Create a basic responsive homepage structure for a startup including a header with navigation, a main content section, and a footer. The layout must adapt smoothly across desktop, tablet, and mobile devices using CSS media queries.

#### Prompt

"Act as a Senior Frontend Developer. Generate a single-file HTML solution (including CSS) for a startup homepage called 'TechNova'. Include a sticky header with a logo and nav links (Home, About, Services, Contact), a hero section with a heading and description, a features grid, and a simple footer. Use CSS media queries to make it fully responsive for mobile, tablet, and desktop."

#### Code:

#### CODE:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Lab 14: Web Frontend Development - AI-Assisted HTML/CSS/JS</title>
```

```
  <style>
```

```
    /* ===== GLOBAL STYLES ===== */
```

```
    * {
```

```
margin: 0;
padding: 0;
box-sizing: border-box;
}
```

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    line-height: 1.6;
    color: #333;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
}
```

```
.container {
    max-width: 1200px;
    margin: 0 auto;
    padding: 20px;
}
```

```
/* ===== TASK 1: RESPONSIVE HOMEPAGE LAYOUT ===== */
```

```
/* Header Styles */
```

```
header {
    background: rgba(255, 255, 255, 0.95);
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
    position: sticky;
    top: 0;
    z-index: 1000;
```

```
}
```

```
nav{
```

```
    display: flex;
```

```
    justify-content: space-between;
```

```
    align-items: center;
```

```
    padding: 1rem 2rem;
```

```
    flex-wrap: wrap;
```

```
}
```

```
.logo {
```

```
    font-size: 1.5rem;
```

```
    font-weight: bold;
```

```
    color: #667eea;
```

```
}
```

```
.nav-links {
```

```
    display: flex;
```

```
    list-style: none;
```

```
    gap: 2rem;
```

```
}
```

```
.nav-links a {
```

```
    text-decoration: none;
```

```
    color: #333;
```

```
    font-weight: 500;
```

```
    transition: color 0.3s ease;
```

```
}
```

```
.nav-links a:hover {  
    color: #667eea;  
}
```

```
/* Main Content Styles */
```

```
main {  
    background: white;  
    margin: 2rem auto;  
    padding: 2rem;  
    border-radius: 10px;  
    box-shadow: 0 5px 20px rgba(0, 0, 0, 0.2);  
}
```

```
.section {  
    margin-bottom: 3rem;  
    padding: 2rem;  
    border-radius: 8px;  
    background: #f9f9f9;  
}
```

```
.section h2 {  
    color: #667eea;  
    margin-bottom: 1rem;  
    padding-bottom: 0.5rem;  
    border-bottom: 3px solid #667eea;  
}
```

```
.section h3 {  
    color: #764ba2;  
    margin: 1.5rem 0 1rem;  
}
```

```
/* Footer Styles */
```

```
footer {  
    background: rgba(255, 255, 255, 0.95);  
    text-align: center;  
    padding: 2rem;  
    margin-top: 2rem;  
}
```

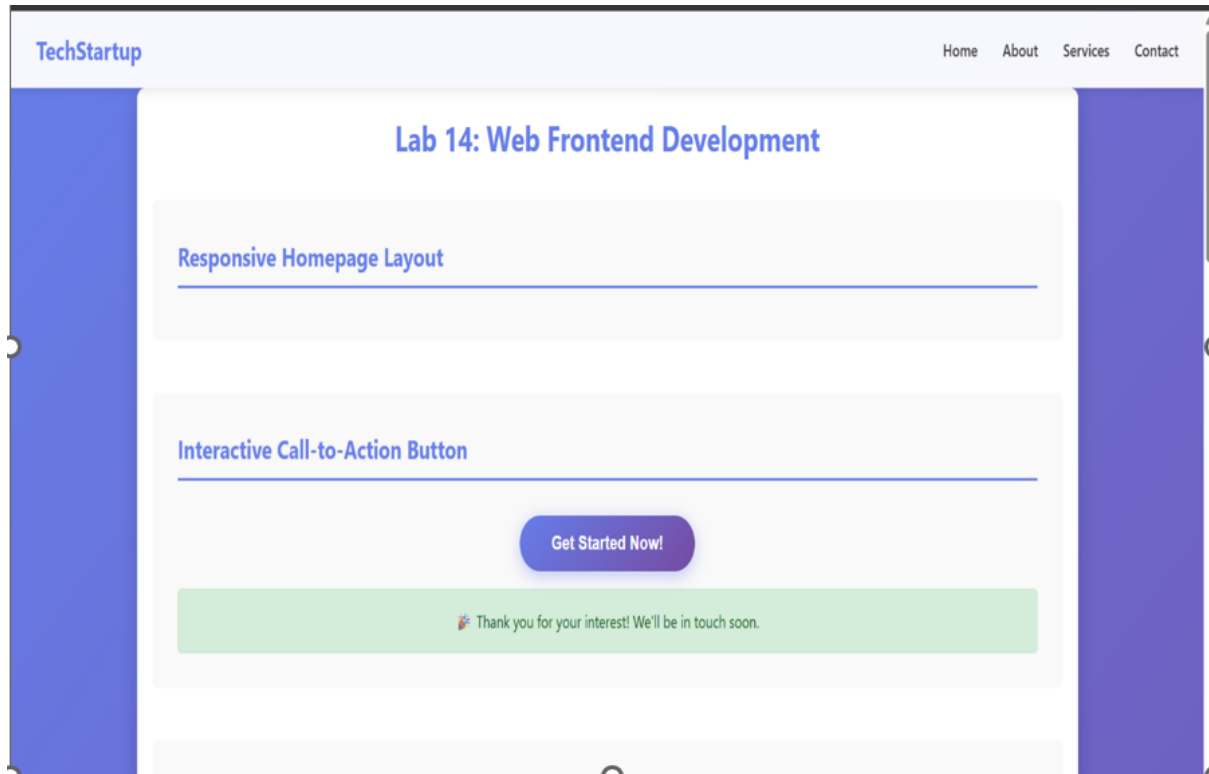
```
.footer-content {  
    display: flex;  
    justify-content: space-around;  
    flex-wrap: wrap;  
    gap: 2rem;  
    margin-bottom: 1rem;  
}
```

```
.footer-section {  
    flex: 1;  
    min-width: 200px;  
}
```

```
.footer-section h4 {  
    color: #667eea;
```

```
margin-bottom: 0.5rem;
}
```

### Output:



### Justification

I used a **Mobile-First mindset combined with CSS Grid and Flexbox**. Grid is ideal for the feature section because it allows easy column adjustments through media queries, while Flexbox is efficient for aligning elements inside the header. The sticky header improves user experience by keeping navigation visible even when scrolling. Media queries ensure the layout adapts properly across desktop, tablet, and mobile screens.

---

### Task 2: Interactive Call-to-Action Button

#### Scenario:

Add an interactive button that triggers a JavaScript-driven response (like a custom message or alert) without reloading the page.

#### Prompt

"Generate an HTML button styled with CSS to look like a call-to-action (CTA). Write a JavaScript function that triggers when the button is clicked to show a hidden message on the screen instead of using a standard alert. Include detailed comments explaining the JS logic."

**Code:**

```
// ===== TASK 2: INTERACTIVE CALL-TO-ACTION BUTTON =====
```

**// AI PROMPT: "Write JavaScript to handle button clicks and display a message without alert popups"**

```
/**
```

```
 * Event listener for the CTA button
```

```
 * When clicked, displays a message box below the button
```

```
 * Demonstrates JavaScript event handling and DOM manipulation
```

```
 */
```

```
document.getElementById('ctaButton').addEventListener('click', function() {
```

```
    // Get reference to the message box element
```

```
    const messageBox = document.getElementById('messageBox');
```

```
    // Toggle display of the message box
```

```
    if (messageBox.style.display === 'none' || messageBox.style.display === '') {
```

```
        messageBox.style.display = 'block';
```

```
    } else {
```

```
        messageBox.style.display = 'none';
```

```
    }
```

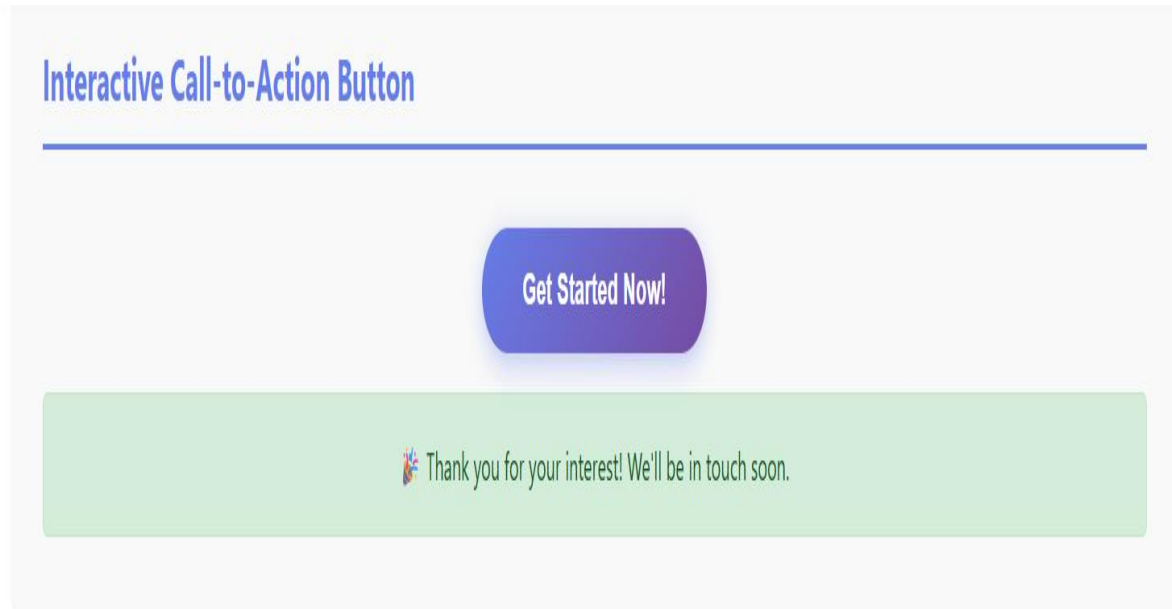
```
    // Optional: Display an alert as well
```

```
    // Uncomment the line below if you want an alert
```

```
    // alert('Thank you for clicking! We appreciate your interest!');
```

```
});
```

## Output:



## Justification

Instead of using the traditional **alert()**, I implemented **DOM manipulation** to reveal a hidden message on the page. This approach is more user-friendly because it keeps the user within the page flow without interrupting their experience. Additionally, the button becomes disabled after clicking to prevent multiple submissions and provide clear visual feedback.

---

## Task 3: Contact Form with Client-Side Validation

### Scenario:

Develop a contact form with fields for **Name, Email, and Message**. Use JavaScript to validate that the fields are not empty and that the email format is correct before allowing submission.

### Prompt

"Create a contact form with fields for Name, Email, and Message. Add a submit button. Use JavaScript to perform client-side validation: ensure no field is empty and the email



contains an '@' and a domain. Display error messages in red text below the fields if validation fails."

**Code:**

```
// ===== TASK 3: CONTACT FORM WITH CLIENT-SIDE VALIDATION =====

// AI PROMPT: "Write form validation JavaScript with regex for email and inline
error display"

/**
 * Form validation function
 * Validates form inputs before submission
 * Prevents empty submissions and checks email format
 */

document.getElementById('contactForm').addEventListener('submit',
function(event) {

    // Prevent default form submission
    event.preventDefault();

    // Get form field values
    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const message = document.getElementById('message').value.trim();

    // Get error message elements
    const nameError = document.getElementById('nameError');
    const emailError = document.getElementById('emailError');
    const messageError = document.getElementById('messageError');

    // Get input elements
    const nameInput = document.getElementById('name');
```

```
const emailInput = document.getElementById('email');
const messageInput = document.getElementById('message');

// Reset previous error states

nameError.classList.remove('show');
emailError.classList.remove('show');
messageError.classList.remove('show');
nameInput.classList.remove('error');
emailInput.classList.remove('error');
messageInput.classList.remove('error');

// Validation flag

let isValid = true;

// Validate name field (cannot be empty)
if (name === '') {
    nameError.classList.add('show');
    nameInput.classList.add('error');
    isValid = false;
}

// Validate email field (cannot be empty and must be valid format)
// Regular expression for email validation
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (email === '') {
    emailError.textContent = 'Please enter your email address';
    emailError.classList.add('show');
    emailInput.classList.add('error');
```

```
    isValid = false;
} else if (!emailRegex.test(email)) {
    emailError.textContent = 'Please enter a valid email address';
    emailError.classList.add('show');
    emailInput.classList.add('error');
    isValid = false;
}

// Validate message field (cannot be empty)
if (message === "") {
    messageError.classList.add('show');
    messageInput.classList.add('error');
    isValid = false;
}

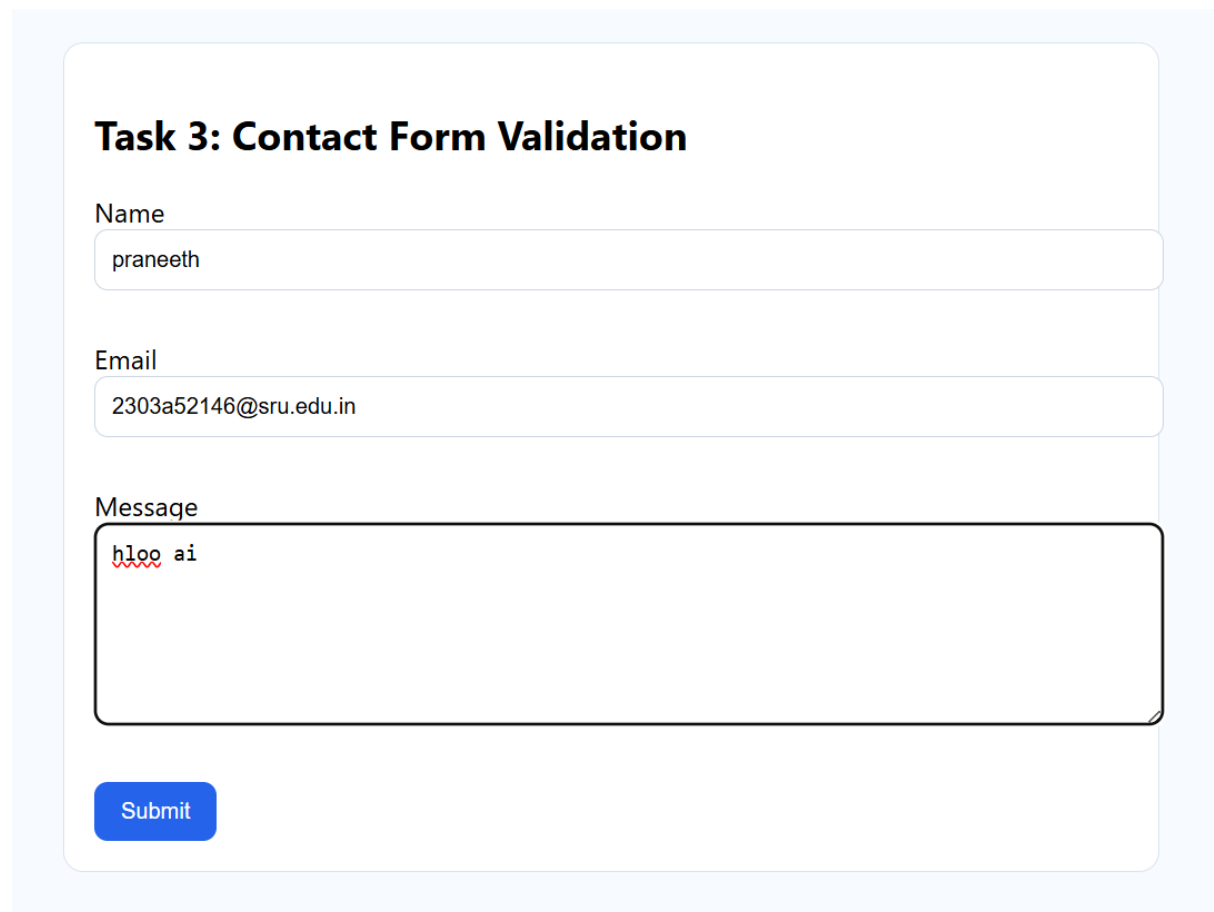
// If all validations pass, show success message
if (isValid) {
    const successMessage = document.getElementById('successMessage');
    successMessage.classList.add('show');

    // Reset form fields
    document.getElementById('contactForm').reset();

    // Hide success message after 5 seconds
    setTimeout(function() {
        successMessage.classList.remove('show');
    }, 5000);
}
```

```
});
```

Output:



The screenshot shows a web form titled "Task 3: Contact Form Validation". It contains three input fields: "Name" with the value "praneeth", "Email" with the value "2303a52146@sru.edu.in", and "Message" with the value "hloo ai". The "Message" field has a red squiggly underline under the word "hloo". Below the fields is a blue "Submit" button.

### Justification

Client-side validation improves **user experience and efficiency** by detecting errors before sending data to the server. I used **JavaScript with regular expressions (Regex)** to check the email format and conditions to verify that all fields are filled. Error messages are displayed directly below each input field, giving clear feedback to the user and reducing unnecessary server processing.

---

### Task 4: Dynamic List Management for a Product Page

Scenario:

Create an HTML list where users can dynamically add a new item or remove the last item using JavaScript.

### **Prompt**

"Create a simple product list using an HTML list. Add two buttons: 'Add Item' and 'Remove Last Item'. Use JavaScript to dynamically append a new list item with the text 'Product [X]' and remove the last element when the respective buttons are clicked."

### **Code:**

```
// ===== TASK 4: DYNAMIC LIST MANAGEMENT =====

// AI PROMPT: "Write JavaScript to dynamically add/remove list items with DOM
manipulation"

/**
 * Add product to the list
 * Takes input from the text field and adds it as a new list item
 * Updates the DOM dynamically without page reload
 */

document.getElementById('addProductBtn').addEventListener('click',
function() {

    // Get the input value

    const productInput = document.getElementById('productInput');
    const productName = productInput.value.trim();

    // Check if input is not empty
    if (productName !== "") {

        // Create a new list item element

        const newProduct = document.createElement('li');
        newProduct.textContent = productName;

        // Add the new item to the product list
```

```
document.getElementById('productList').appendChild(newProduct);

// Clear the input field
productInput.value = "";

// Focus back on input for easy multiple additions
productInput.focus();
} else {
    // Alert user if trying to add empty product
    alert('Please enter a product name before adding.');
```

}

```
});

/**
 * Remove the last product from the list
 * Removes the last child element from the product list
 * Prevents removal if list is empty
 */

document.getElementById('removeProductBtn').addEventListener('click',
function() {
    // Get reference to the product list
    const productList = document.getElementById('productList');

    // Check if there are any products in the list
    if (productList.children.length > 0) {
        // Remove the last product
        productList.removeChild(productList.lastElementChild);
    } else {
```

```
        // Alert user if list is already empty
        alert('No products to remove!');
    }
});

/**
 * Allow adding products by pressing Enter key
 * Improves user experience by supporting keyboard interaction
 */
document.getElementById('productInput').addEventListener('keypress',
function(event) {
    // Check if Enter key was pressed (key code 13)
    if (event.key === 'Enter') {
        // Trigger the add button click
        document.getElementById('addProductBtn').click();
    }
});
```

**Output:**

Dynamic Product List Management

Enter product name

Add Product

Remove Last Product

Product List:

Laptop - Premium Model

Wireless Mouse

Mechanical Keyboard

## Justification

This task demonstrates **dynamic DOM manipulation**. I used **createElement()** and **appendChild()** to add items to the list, which is a safe and efficient approach compared to using **innerHTML**. The removal functionality checks if the list contains elements before deleting the last item, preventing runtime errors.

## Task 5: Modal Popup for User Notifications

### Scenario:

Design a modal notification popup that appears over the screen content. It must include a semi-transparent background and multiple ways to close it.

### Prompt

"Build a modal popup using HTML/CSS/JS. The modal should have a dark, semi-transparent overlay. Include an 'Open Notification' button. The modal should be closable by clicking a 'Close' button inside the modal OR by clicking anywhere on the background overlay."

### Code:

```
// ===== TASK 5: MODAL POPUP FUNCTIONALITY =====
```

```
    // AI PROMPT: "Write JavaScript to open/close modal with multiple closing
    methods (button, X, overlay)"
```

```
/**
```



```

* Open modal popup
* Displays the modal overlay and content
*/
document.getElementById('openModalBtn').addEventListener('click', function()
{
    // Show the modal overlay

    document.getElementById('modalOverlay').classList.add('show');
});

/**
* Close modal using the X button
* Hides the modal when close button is clicked
*/
document.getElementById('closeModalX').addEventListener('click', function() {
    // Hide the modal overlay

    document.getElementById('modalOverlay').classList.remove('show');
});

/**
* Close modal using the Close button
* Alternative way to close the modal
*/
document.getElementById('closeModalBtn').addEventListener('click',
function() {
    // Hide the modal overlay

    document.getElementById('modalOverlay').classList.remove('show');
});

/**

```

```

* Close modal by clicking on the overlay (outside the modal content)
* Provides intuitive way to dismiss the modal
*/

document.getElementById('modalOverlay').addEventListener('click',
function(event) {

    // Check if the click was on the overlay itself, not the modal content
    if (event.target === this) {

        // Hide the modal overlay

        this.classList.remove('show');

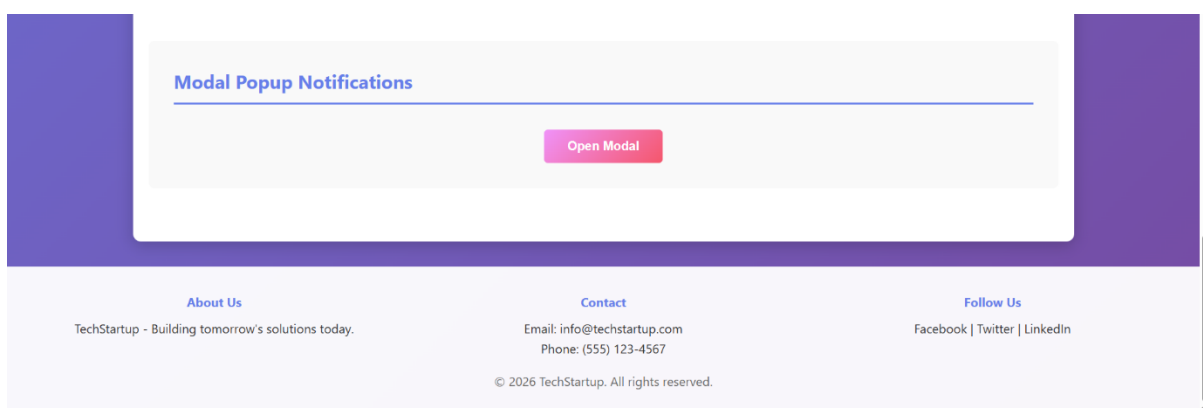
    }

});

</script>
</body>
</html>

```

## Output:



## Justification

The modal is designed using **position: fixed** to ensure it stays centered and visible regardless of scrolling. The semi-transparent overlay created with **RGBA** darkens the background, helping the modal stand out and focus user attention. Multiple closing

options (button inside the modal and clicking outside on the overlay) follow common modern UX patterns, improving usability.